

Project Title

Hospital Management System

Course: Database Management Systems (DBMS)

Team Members

- 1 Bhumireddy**Tharun**
S20240010032
- 2 Maddukuri**Santosh**
S20240010130
- 3 D.Naga**Varshith**
S20240010063
- 4 **Rakshitha**Badugu
S20240010024
- 5 Karanam**Raghuveer**
S20240010107

Problem Statement:

Hospitals handle large volumes of data daily, including patients, doctors, appointments, lab results, and billing information. Manual or poorly structured data management often leads to inefficiency, human error, and difficulty in tracking performance and payments.

To address this, the Hospital Management System Database is developed to automate and streamline hospital operations. It manages patient visits, admissions, and billing while enforcing role-based access for staff. The system also calculates doctor efficiency based on feedback, visit duration, and case volume, ensuring data-driven performance evaluation and better service management.

Using SQL functions, procedures, and triggers, the system maintains integrity, accuracy, and automation across all hospital activities.

Objectives:



To design a centralized, normalized database for managing patients, staff, appointments, and billing.



To automate billing and ensure real-time updates during patient visits using triggers.



To implement role-based operations, allowing only authorized staff (e.g., nurses, lab technicians) to perform specific tasks.



To calculate and monitor doctor efficiency using stored procedures and feedback data.



To generate analytical insights, such as unpaid bill reports and active patient counts, for better hospital decision-making.



To enhance data accuracy, consistency, and performance tracking through SQL-based automation.

Requirement Analysis:

The Hospital Management System (HMS) aims to automate and manage hospital operations including patient registration, staff management, appointments, visits, lab testing, billing, and doctor performance evaluation. This section analyzes the functional and non-functional requirements of the proposed database system.

Functional Requirements:

Patient Management:

- Store and manage patient details including personal information, contact details, and medical history.
- Allow tracking of both outpatient and inpatient records.

Staff & Doctor Management:

- Maintain records of all hospital staff with role differentiation (Doctor, Nurse, Lab Technician, Receptionist, Admin).
- Store doctors' profiles, including specialization, experience, and consultation fees.

Appointment & Visit Handling:

- Allow patients to schedule appointments linked with doctors and departments.
- Track each visit type (Outpatient / Inpatient) along with check-in and check-out times.
- Handle inpatient admissions with room details and admission reasons.

Vitals & Lab Operations:

- Permit only authorized nurses to record vitals (via triggers).
- Allow only doctors to order lab tests and lab technicians to collect and report results.
- Auto-update lab order status when results are entered.

Patient Management :

- Store and manage patient details including personal information, contact details, and medical history.
- Allow tracking of both outpatient and inpatient records.

Staff & Doctor Management:

- Maintain records of all hospital staff with role differentiation (Doctor, Nurse, Lab Technician, Receptionist, Admin).
- Store doctors' profiles, including specialization, experience, and consultation fees.

Appointment & Visit Handling:

- Allow patients to schedule appointments linked with doctors and departments.
- Track each visit type (Outpatient / Inpatient) along with check-in and check-out times.
- Handle inpatient admissions with room details and admission reasons.

Vitals & Lab Operations:

- Permit only authorized nurses to record vitals (via triggers).
- Allow only doctors to order lab tests and lab technicians to collect and report results.
- Auto-update lab order status when results are entered.

Billing Management:

- Automatically create bills for each visit using triggers.
- Add consultation fees and additional charges dynamically through stored procedures.
- Track payment status (Paid / Unpaid / Partially Paid) and generate unpaid bill reports.

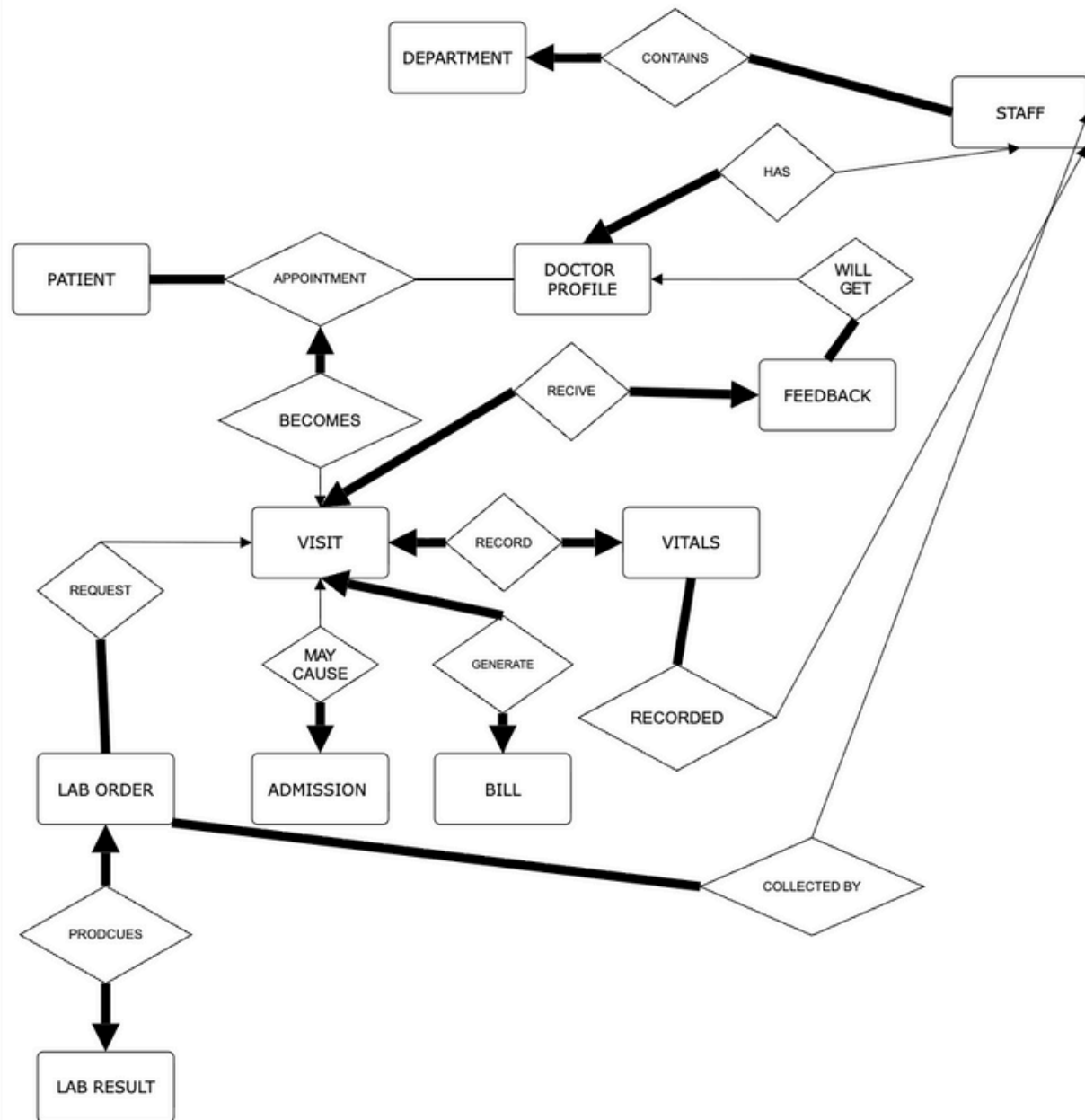
Doctor Efficiency Evaluation:

- Automatically calculate and update doctor efficiency scores based on visit duration, feedback, and patient count.
- Allow performance tracking and ranking of doctors.

Feedback & Reporting:

- Collect patient feedback after each visit.
- Generate analytical reports such as active unpaid patients, total visits, and efficiency summaries.

ER Diagram :



Attributes:

1. Patient :

patient_id — Atomic
name — Composite
dob — Atomic
age — Derived
gender — Atomic
phone — Multivalued
email — Multivalued
address — Composite

2. Department

dept_id — Atomic
name — Atomic
description — Atomic
head_staff_id — Atomic

3. Staff

staff_id — Atomic
name — Composite
role — Atomic
phone — Multivalued
email — Multivalued
active — Atomic
hired_date — Atomic
dept_id — Atomic

4. DoctorProfile

doctor_id — Atomic
license_number — Atomic
years_experience — Atomic
consultation_fee — Atomic
office_room — Atomic
primary_specialization — Atomic
efficiency_score — Derived

5. Appointment

appointment_id — Atomic
patient_id — Atomic
doctor_id — Atomic
scheduled_at — Atomic
status — Atomic
reason — Atomic
preferred_dept — Atomic
booked_by — Atomic

6. Visit

visit_id — Atomic
appointment_id — Atomic
patient_id — Atomic
doctor_id — Atomic
visit_type — Atomic
checkin_time — Atomic
checkout_time — Atomic
duration_minutes — Derived
diagnosis — Atomic

7. Admission

admission_id — Atomic
admission_number — Atomic
visit_id — Atomic
patient_id — Atomic
doctor_id — Atomic
admission_time — Atomic
discharge_time — Atomic
room_number — Atomic
admission_reason — Atomic

8. Vitals

vitals_id — Atomic
visit_id — Atomic
recorded_by — Atomic
recorded_at — Atomic
temperature — Atomic
pulse — Atomic
systolic — Atomic
diastolic — Atomic
spo2 — Atomic
respiratory_rate — Atomic

9. LabOrder

lab_order_id — Atomic
visit_id — Atomic
ordered_by — Atomic
collected_by — Atomic
test_name — Multivalued
status — Atomic
ordered_at — Atomic
collected_at — Atomic

10. LabResult

lab_result_id — Atomic
lab_order_id — Atomic
reported_by — Atomic
result — Atomic
units — Atomic
reference_range — Atomic
reported_at — Atomic

11. Bill

bill_id — Atomic
visit_id — Atomic
patient_id — Atomic
total_amount — Derived
status — Atomic

12. Feedback

feedback_id — Atomic
visit_id — Atomic
patient_id — Atomic
score — Atomic
comment — Atomic

Justification of Selected Users

The users in the Hospital Management System are selected based on their roles in real hospital operations, ensuring accountability, security, and task specialization.



Doctor:

Core medical professional responsible for patient diagnosis, lab test requests & treatment. Efficiency is auto-calculated from feedback and visit data.



Nurse:

Handles patient vitals and assists doctors. Only nurses can record vitals to ensure accurate medical tracking.



Lab Technician:

Performs lab tests, records results, and updates lab orders. Ensures test integrity and compliance with medical protocols.



Receptionist:

Manages patient registration & appointment scheduling, serving as the primary link between patients and hospital operations.



Admin:

Oversees overall database control, verifies operations, and manages staff and department-level configurations.

Normalization Steps:

1 Unnormalized Form (UNF)

Initially, all data (patients, doctors, visits, bills) were stored together, causing duplication and update anomalies.

Fix: Divided data into separate tables — Patient, DoctorProfile, Department, Appointment, Visit, Bill, and Feedback.

2 First Normal Form (1NF)

Removed repeating groups and ensured atomic values.

Fix: Added primary keys and created separate tables for unique entities like Doctor and Department.

3 Second Normal Form (2NF)

Removed partial dependencies.

Fix: Moved doctor-specific details to DoctorProfile and linked through foreign keys.

4 Third Normal Form (3NF)

Removed transitive dependencies.

Problem: patient_id in Bill and Feedback depended on visit_id.

Fix: Removed redundant columns and derived patient details through Visit → Appointment → Patient.

5 Boyce-Codd Normal Form (BCNF)

Eliminated non-superkey dependencies and derived attributes.

Fix Summary:

- Visit: Removed redundant patient_id, doctor_id, and derived duration_minutes.
- Admission: Removed redundant patient_id and doctor_id.
- Bill & Feedback: Removed unnecessary patient_id dependency.

Database Schema:

Table Structures:

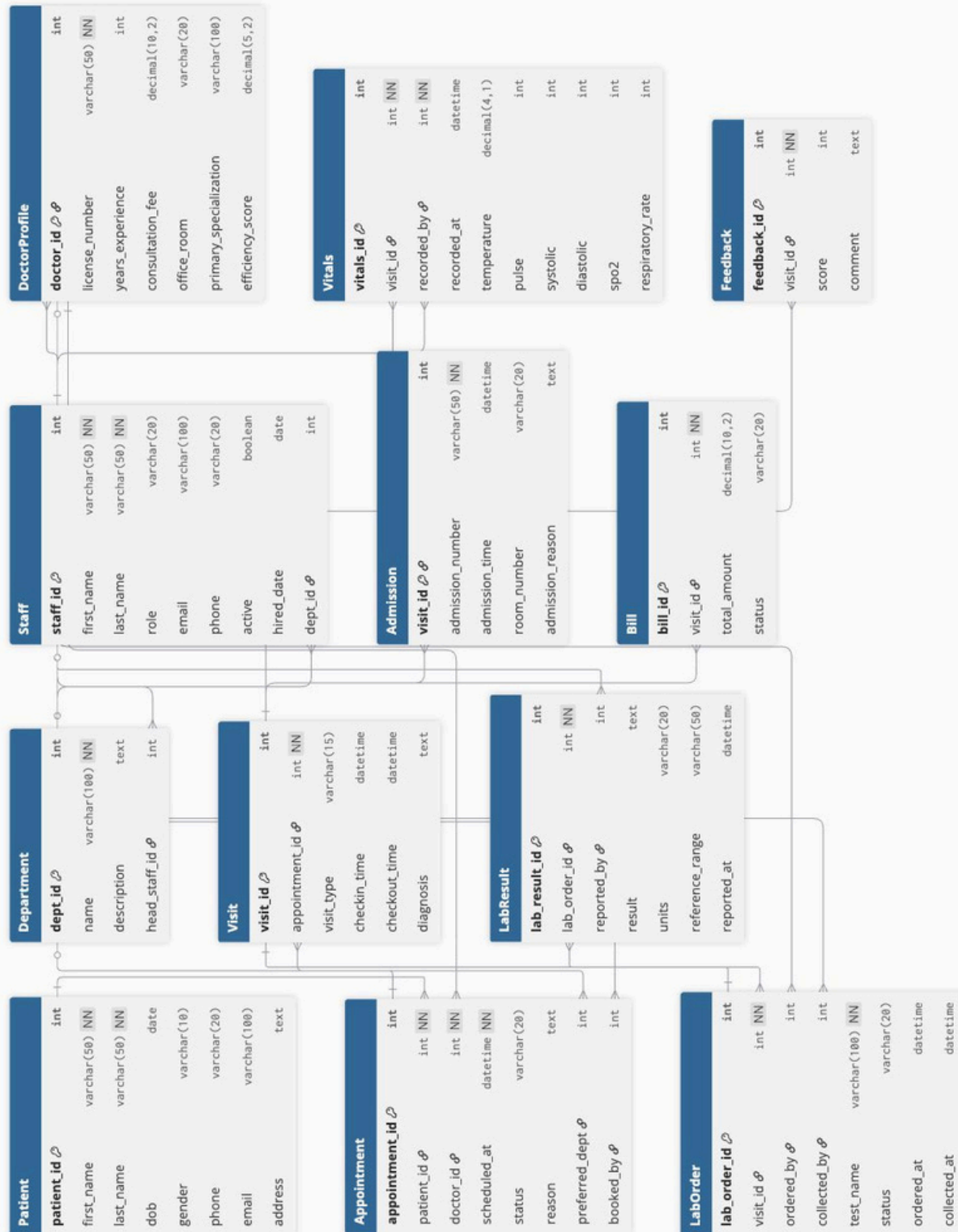


Table Setup Codes:

Due to the number of SQL scripts used for creating and initializing the database tables, they have not been fully included in this report. The complete set of SQL table creation codes can be accessed through the GitHub repository link provided below.

Github link:

<https://github.com/Rag-2007/DBMS-PROJECT/tree/main/CODES>

Output Screenshots:

Due to the large number of output screenshots generated during testing and execution, they could not all be included in this report. To view the complete set of outputs and project demonstrations, please refer to the GitHub repository link provided below.

Github link:

<https://github.com/Rag-2007/DBMS-PROJECT/tree/main/OUTPUTS>

Conclusion and Future Scope:

Conclusion

The Hospital Management System (HMS) was successfully developed to organize and automate hospital operations such as patient records, staff roles, appointments, visits, billing, and lab activities. The system ensures data accuracy, security, and role-based access, allowing each staff member to perform only their specific duties. The inclusion of a Doctor Efficiency module makes this project unique by enabling hospitals to assess doctor performance using feedback and activity data. Overall, it provides a reliable, scalable, and well-structured database solution for efficient healthcare management.

Future Scope

In the future, this system can be enhanced by integrating a web or mobile interface for real-time access, implementing AI-based analytics for predicting workloads, and adding automated notifications for patients. It can also be extended with EHR support and data dashboards to offer deeper insights and improve hospital decision-making.

Contribution of Each Group Member:

The project was successfully completed as a collaborative effort by all group members. Each member contributed to different modules and phases of development, ensuring a complete and efficient Hospital Management System. The contributions are as follows:

Maddukuri**Santosh**:

- Handled Requirement Analysis and Problem Statement & Objectives sections.
- Took responsibility for testing and verification of all modules to ensure accuracy and correctness of database operations.

Bhumireddy**Tharun**:

- Designed and developed the ER Diagram and provided the Justification for Selected Users.
- Ensured the overall design aligned with the project objectives and represented real-world hospital workflows effectively.

D.Naga**Varshith**:

- Worked on the Relational Schema Design and contributed to the Justification of Users section.
- Focused on maintaining normalization and ensuring data consistency between entities.

RakshithaBadugu:

- Implemented Table Creation in SQL.
- Performed Normalization of tables to eliminate redundancy and maintain data integrity.
- Prepared the documentation for this project.

Karanam**Raghuveer**:

- Developed all Queries, Procedures, Functions, and Triggers used in the system.
- Managed Output Screenshots, GitHub version control, and the demonstration aspects of the project.